

Reviewer Pack — Minimal Frobenius Norm and Communication Complexity Rank Cor...

Entry 1513d623a849 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 18:23:15 UTC

Minimal Frobenius Norm and Communication Complexity Rank Correlation

Entry ID: 1513d623a849 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 18:23:15 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (specifically the theory of schemes) **Field B** (complexity object): Communication Complexity

Statement:

For every N -ary boolean function f , the minimal Frobenius norm ($\text{Frobenius}(f)$) of its associated variety V_f in projective space is linearly correlated with its communication complexity rank $r(f)$, such that $\text{Frobenius}(f) = \Theta(r(f))$. Equivalently: for all instances with property P , property Q holds where P is defined as ‘the variety V_f has minimal Frobenius norm among all varieties associated with N -ary boolean functions of the same communication complexity rank’, and Q is defined as ‘the communication complexity rank $r(f) = \Theta(\text{Frobenius}(f))$ ’.

Rationale (proposer’s reasoning):

The algebraic structure of the scheme associated with a boolean function provides a geometric representation that could potentially reveal underlying complexities in the communication process. The Frobenius norm, which measures the ‘size’ of the variety, might act as an indicator for the communication complexity rank, reflecting the intricacy of information distribution between parties.

Taxonomy category: algebraic_geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 77a50851a23893b1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 95% of generated N -ary boolean functions have a Pearson correlation coefficient between their Frobenius norm and communication complexity rank greater than or equal to 0.7, with no seed producing a correlation below 0.6.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Frobenius norm" AND "communication complexity rank" IN title - "variety V_f" AND minimal Frobenius norm AND communication complexity" - "linear correlation" AND "algebraic geometry schemes" AND "communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.6s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity(f):
        n = len(f)
        max_comm_cost = 0
        for i in range(2**n):
            comm_cost = sum(abs(f[i] - f[j]) for j in range(i+1, 2**n)) / (2**(n-1))
            if comm_cost > max_comm_cost:
                max_comm_cost = comm_cost
        return max_comm_cost

    def frobenius_norm(V):
        n = len(V)
        sum_of_squares = sum(sum(x**2 for x in row) for row in V)
        return math.sqrt(n * sum_of_squares)

    def generate_variety(f):
        n = len(f)
        variety = []
        for i in range(2**n):
            if all(f[i] == f[j] for j in range(i+1, 2**(n-1))):
                variety.append([f[i]] * (2**(n-1)))
```

```

    return variety

def pearson_correlation(X, Y):
    n = len(X)
    mean_X = sum(X) / n
    mean_Y = sum(Y) / n
    numerator = sum((X[i] - mean_X) * (Y[i] - mean_Y) for i in range(n))
    denominator = math.sqrt(sum((X[i] - mean_X)**2 for i in range(n))) * math.sqrt(sum((Y[i] - mean_Y)**2 for i in range(n)))
    return numerator / denominator if denominator != 0 else 0

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_boolean_function(n)
    comm_rank = communication_complexity(f)
    V = generate_variety(f)
    frobenius_norm_value = frobenius_norm(V)
    results.append((comm_rank, frobenius_norm_value))

X, Y = zip(*results)
correlation_coefficient = pearson_correlation(X, Y)

return {
    "metric_name": "Pearson Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.7,
    "counterexample": "" if correlation_coefficient >= 0.7 else f"Correlation coefficient {correlation_coefficient}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd4d650d.py", line 82, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd4d650d.py", line 59, in run_trial
    comm_rank = communication_complexity(f)
               ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd4d650d.py", line 28, in communication
    comm_cost = sum(abs(f[i] - f[j]) for j in range(i+1, 2*n)) / (2*(n-1))
               ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dd4d650d.py", line 28, in <genexpr>
    comm_cost = sum(abs(f[i] - f[j]) for j in range(i+1, 2*n)) / (2*(n-1))
                   ~~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16091
2	preregistration	ollama_remote	glm4:latest	0	0	9161
3	novelty	ollama_remote	glm4:latest	0	0	17956
4	novelty	ollama_remote	glm4:latest	0	0	17645
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16859
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20578
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	230542
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20650
9	judge	ollama_remote	glm4:latest	0	0	29680

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 379161 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/1513d623a849.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1513d623a849.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1513d623a849.tar.gz (if generated)